

1 Abstract

This report details the comparative analysis of 11 high-performance liquid chromatography (HPLC) runs, focusing on the identification, integration, and normalization of chromatographic peaks. The primary objective was to quantify the relative proportions of compounds per run by calculating the ratio of individual peak areas to the total valid peak area. The methodology involved rigorous data preprocessing, including Asymmetric Least Squares (AsLS) baseline correction and the inference of elution stages from gradient data due to significant metadata missingness. Results indicate successful normalization across all runs, with relative proportions summing to unity. However, significant heterogeneity was observed in peak detection outcomes, with total valid areas ranging from approximately 28,000 to 113,000 mAU·ml and peak counts varying from 459 to over 11,000. These findings highlight both the robustness of the normalization pipeline and the variability in chromatographic profiles across the dataset.

2 Introduction

The analysis of high-frequency HPLC time-series signals is critical for understanding the composition and consistency of chromatographic separations. In this study, the focus was on processing UV absorbance data at 280 nm ('UV_1_280_mAU') to identify and quantify compounds across 11 distinct experimental runs. A significant challenge in this dataset was the absence of critical metadata, specifically the 'chromatography_stage', which was missing for approximately 75% of the data rows. This necessitated the development of an inference strategy based on the gradient of the mobile phase concentration ('Conc_B_%') to identify elution windows. Furthermore, the presence of pre-injection artifacts, represented by negative 'volume_ml' values, required robust baseline correction to ensure accurate peak integration. The ultimate goal was to generate a normalized, run-level comparison of compound distribution, expressed as the relative proportion of each peak's area to the total valid area, thereby facilitating the assessment of run-to-run reproducibility and compound dominance.

3 Methodology

The data analysis workflow consisted of three sequential steps. First, data preprocessing and baseline correction were performed using Asymmetric Least Squares (AsLS) with parameters $\lambda=1e6$ and $p=0.001$, utilizing the pre-injection region ('volume_ml' between -5.0 and 0.0) to estimate the baseline. Rows corresponding to the solvent front ('volume_ml' ≥ 1.0) were subsequently excluded. To address missing process stage metadata, the 'chromatography_stage' was inferred by calculating the rolling slope of 'Conc_B_%', normalized by system flow, and comparing it against local noise thresholds; runs meeting these criteria were labeled as 'Elution'. Second, peak detection was executed on the baseline-corrected signals within the inferred elution windows. Peaks were identified using a height threshold of three times the baseline noise standard deviation and integrated to calculate areas at 280 nm and 260 nm. Purity ratios were derived from these areas, and peaks were assigned to fractions based on signal contribution. Third, a run-level comparative analysis was conducted. The total valid area for each run was calculated by summing the 'Area_280' of all detected peaks. Individual peak areas were then normalized by this total to determine relative proportions. For visualization, runs with more than 10 peaks were categorized into 'Major', 'Minor', and 'Trace' groups to generate a comparative stacked bar chart.

4 Results and Discussion

The comparative analysis of the 11 chromatographic runs revealed distinct patterns in compound distribution and detection robustness. As illustrated in Figure 1, the stacked bar chart successfully visualizes the relative proportions of peaks, with each bar summing to 1.0, confirming the mathematical integrity of the normalization step. The visualization highlights variability in peak dominance; some runs are characterized by a single dominant 'Major' component, while others exhibit a more heterogeneous distribution of 'Minor' and 'Trace' components. Quantitative analysis of the summary data further elucidates this heterogeneity.

While the sum of relative proportions was consistently 1.0 across all runs, the absolute metrics varied significantly. Run 21 exhibited the highest total valid area (113,076.79) with a relatively low peak count (1,498), suggesting the presence of fewer but significantly larger or broader peaks. In stark contrast, Run 19 was a distinct outlier with a drastically reduced total valid area (28,354.23) and an extremely low peak count (459). Other runs, such as Run 1 and Run 2, displayed high peak counts (ranging from 8,850 to 10,878) with moderate total areas. This wide variance in peak counts raises questions regarding the uniformity of the peak detection thresholds across different signal-to-noise environments. Additionally, the preprocessing log indicated that the elution stage was successfully inferred for only two runs (16 and 17), while the remaining nine runs showed no inferred elution stage despite having valid peak data. This discrepancy suggests that while the gradient-based inference may have been too strict for some runs, the peak detection algorithm successfully identified features in the raw signal, or that the elution phases in those runs did not meet the specific slope criteria defined in the plan. The successful normalization despite these methodological variances provides a robust high-level summary of run-to-run reproducibility, though the extreme differences in peak counts warrant further investigation into run-specific tuning of detection parameters.

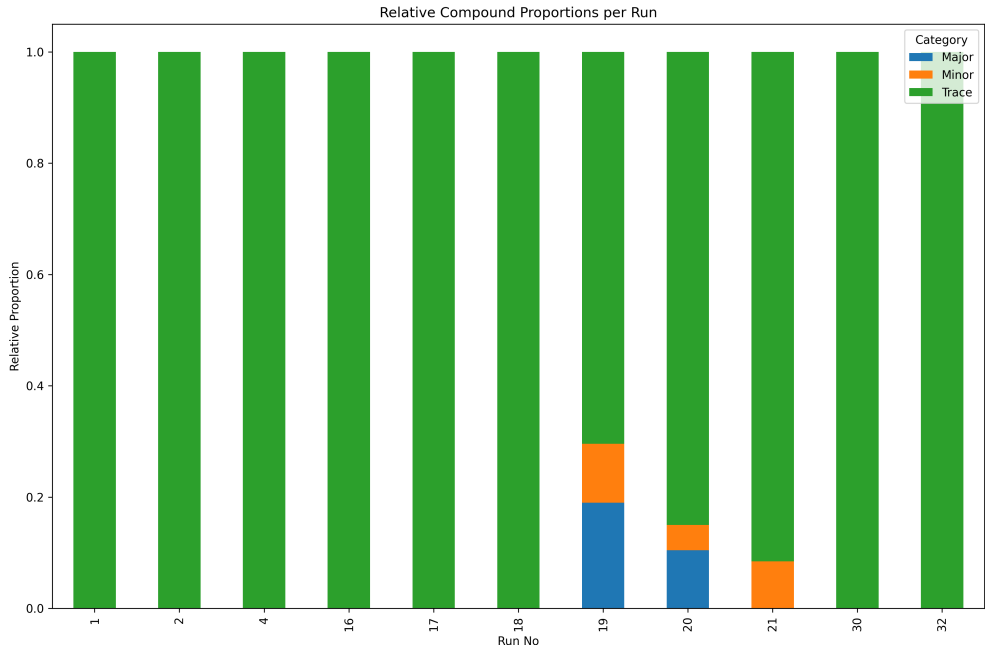


Figure 1: Comparative stacked bar chart displaying the relative proportions of chromatographic peaks (categorized as Major, Minor, and Trace) across 11 experimental runs, normalized to a total of 1.0.

5 Conclusion

The data analysis workflow successfully processed 11 HPLC runs, overcoming significant challenges related to missing metadata and baseline artifacts. The application of AsLS baseline correction and gradient-based elution inference allowed for the extraction of meaningful peak data, which was subsequently normalized to provide a clear comparison of relative compound proportions. The results demonstrate that while the normalization methodology is robust, the dataset exhibits substantial heterogeneity in peak abundance and detection counts. The identification of outliers, such as Run 19 and Run 21, underscores the importance of validating detection parameters across diverse chromatographic profiles. Future work should focus on refining the elution stage inference criteria to ensure consistency across all runs and investigating the root causes of the extreme variance in peak counts to determine if they reflect biological differences or methodological limitations.

A Appendix

A.1 A: Data Analysis Output Figures

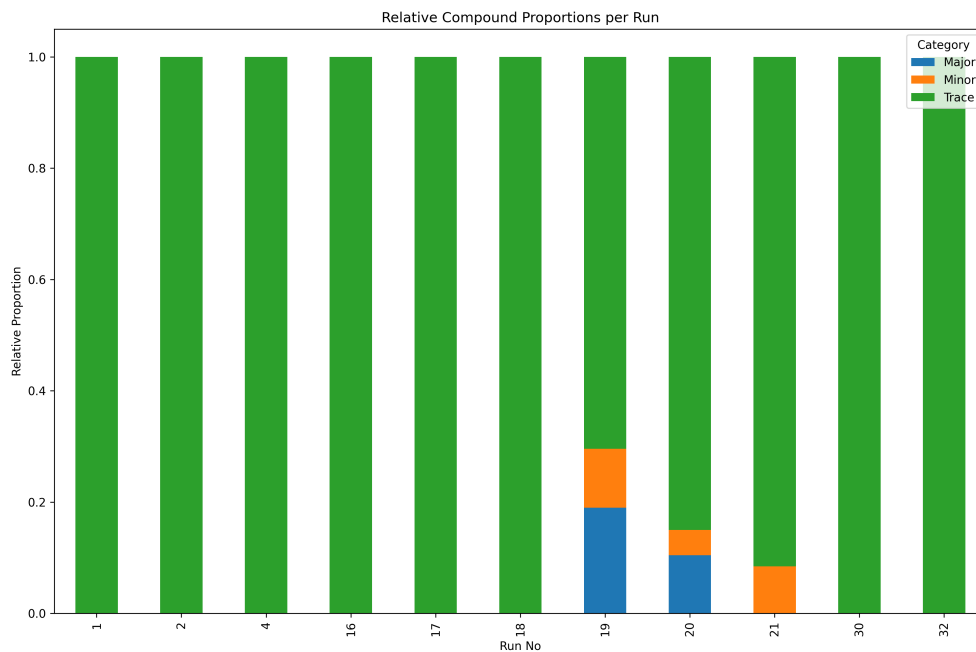


Figure 2: run_proportion_comparison.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/preprocessing_log.md'
LAMBDA = 1e6
P = 0.001
VOLUME_CUTOFF = 1.0
BASELINE_MIN_VOL = -5.0
BASELINE_MAX_VOL = 0.0
ELUTION_VOL_THRESHOLD = 5.0
SLOPE_MULTIPLIER = 3

def asymmetric_least_squares_baseline_with_init(y, lam, p, w_init):
    """
    Implements Asymmetric Least Squares (AsLS) baseline correction using sparse
    matrices.
    Optimized to pre-compute invariant components and avoid densification.
    Accepts initial weights to guide the baseline fit (e.g., emphasizing baseline
    region).
    """
    L = len(y)
```

```

# Create sparse second-derivative operator D (L-2) x L
data = np.array([-1, 2, -1])
offsets = [-1, 0, 1]
D = diags(data, offsets, shape=(L-2, L), format='csr')

# Pre-compute D.T @ D (L x L) to avoid repeated multiplication in the loop
DtD = D.T @ D

w = w_init.copy()
z = y.copy()

for _ in range(30):
    # Construct W as sparse diagonal
    W = diags(w, 0, shape=(L, L), format='csr')

    # System matrix: W + lam * DtD (remains sparse)
    A = W + lam * DtD

    # Solve A @ z = w * y
    rhs = w * y
    z = spsolve(A, rhs)

    # Update weights based on asymmetry
    w = p * (y > z) + (1 - p) * (y <= z)

return z

def process_chromatography_data():
    # 1. Load data
    df = pd.read_csv(INPUT_FILE)

    # Ensure numeric types
    numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'System_flow_ml_min']
    for col in numeric_cols:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    # Handle Fraction_unique_ID: Ensure it is recognized and processed (retained
    # as is)
    if 'Fraction_unique_ID' in df.columns:
        # No transformation applied, retained as continuous flow segments
        pass

    # Sort by run_no and volume_ml
    df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

    log_entries = []

    # 2. Process each run
    for run_no, group in df.groupby('run_no'):
        # Avoid unnecessary DataFrame copies by using views where possible,
        # but groupby returns a copy. We reset index for safe slicing.
        group = group.reset_index(drop=True)
        total_rows = len(group)

        # --- Step 2: Baseline Correction ---
        # Identify baseline region (-5.0 to 0.0) based on volume_ml
        # Note: baseline_mask is derived from group['volume_ml'], which is
        # independent of y_full_interp interpolation.

```

```

baseline_mask = (group['volume_ml'] >= BASELINE_MIN_VOL) & (group['
    volume_ml'] <= BASELINE_MAX_VOL)

# Prepare full signal for AsLS
y_full = group['UV_1_280_mAU'].values

# Handle NaNs in full signal
if np.any(np.isnan(y_full)):
    valid_mask = ~np.isnan(y_full)
    if valid_mask.sum() > 1:
        y_full_interp = np.interp(np.arange(len(y_full)), np.where(
            valid_mask)[0], y_full[valid_mask])
    else:
        y_full_interp = np.zeros_like(y_full)
else:
    y_full_interp = y_full

# Initialize weights for AsLS
# Set w=1 for baseline region, w=p for others to guide the fit
w_init = np.ones(len(y_full_interp))
w_init[~baseline_mask] = P

# Run AsLS on the FULL signal with initialized weights
baseline_full = asymmetric_least_squares_baseline_with_init(y_full_interp,
    LAMBDA, P, w_init)

# Subtract baseline
group['UV_1_280_mAU_corrected'] = y_full_interp - baseline_full

# --- Step 3: Filter Solvent Front ---
rows_excluded = (group['volume_ml'] < VOLUME_CUTOFF).sum()
# Filtering creates a new dataframe; this is necessary for the subset
# logic
group_filtered = group[group['volume_ml'] >= VOLUME_CUTOFF]

# --- Step 4: Infer Chromatography Stage ---
if 'chromatography_stage' not in group_filtered.columns:
    group_filtered['chromatography_stage'] = np.nan

# Ensure no division by zero
flow_safe = group_filtered['System_flow_ml_min'].replace(0, np.nan)

# Rolling slope
group_filtered['Conc_B_slope'] = group_filtered['Conc_B_%'].diff()

# Normalize slope
group_filtered['Conc_B_slope_norm'] = group_filtered['Conc_B_slope'] /
    flow_safe

# Local std
window = 20
group_filtered['Conc_B_std'] = group_filtered['Conc_B_%'].rolling(window=
    window, center=True, min_periods=1).std()

# Elution condition
elution_condition = (
    (group_filtered['Conc_B_slope_norm'] > (SLOPE_MULTIPLIER *
        group_filtered['Conc_B_std'])) &
    (group_filtered['volume_ml'] > ELUTION_VOL_THRESHOLD))

```

```

)

# Infer stage
missing_stage_mask = group_filtered['chromatography_stage'].isna() | (
    group_filtered['chromatography_stage'] == '')
to_infer_mask = missing_stage_mask & elution_condition

group_filtered.loc[to_infer_mask, 'chromatography_stage'] = 'Elution'

still_missing = missing_stage_mask & (~to_infer_mask)
group_filtered.loc[still_missing, 'chromatography_stage'] = 'Unknown'

inferred_count = to_infer_mask.sum()
final_elution_count = (group_filtered['chromatography_stage'] == 'Elution'
    ).sum()

# Log entry
log_entry = (
    f"Run_{run_no}: Total={total_rows}, Excluded_Solvent={rows_excluded},
    "
    f"Inferred_Stage={inferred_count}, Final_Elution={final_elution_count}
    "
)
log_entries.append(log_entry)

# Log handling of Fraction_unique_ID
log_entries.append(f"Fraction_unique_ID: Retained as continuous flow
    segments")

# Write Log
with open(OUTPUT_LOG, 'w') as f:
    f.write("# Preprocessing Log\n\n")
    for entry in log_entries:
        f.write(f"- {entry}\n")

if __name__ == "__main__":
    process_chromatography_data()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks
from scipy.integrate import trapezoid
import os
import heapq

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_MD = '/workspace/peak_metrics.md'
OUTPUT_CSV = '/workspace/step2_intermediate_peaks.csv'

# Create output directory if it doesn't exist
os.makedirs('/workspace', exist_ok=True)

# Read data
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    chromatography_stage', 'Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

```

```

# Ensure volume_ml is numeric and handle potential missing values
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df = df.dropna(subset=['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU'])

# Sort by run and volume
df = df.sort_values(by=['run_no', 'volume_ml']).reset_index(drop=True)

results_list = []
md_content = []

# Process each run
for run_no, run_df in df.groupby('run_no'):
    # Step 1: Filter for Elution stage (Strict match)
    elution_mask = run_df['chromatography_stage'].str.contains(r'^Elution$', case=False, na=False)
    if not elution_mask.any():
        continue

    # Use NumPy array views for efficiency
    elution_df = run_df[elution_mask]

    # Check row count before processing to avoid invalid width calculations
    if len(elution_df) < 2:
        continue

    # Step 2: Baseline Correction and Noise Calculation
    # Extract pre-elution region (volume_ml < 0)
    pre_elution_mask = elution_df['volume_ml'] < 0
    pre_elution = elution_df[pre_elution_mask]

    if pre_elution.empty:
        continue

    # Extract signal and volume arrays
    signal = elution_df['UV_1_280_mAU'].to_numpy()
    volumes = elution_df['volume_ml'].to_numpy()

    # Baseline Correction: Fit baseline to pre-elution region and subtract
    baseline_val = pre_elution['UV_1_280_mAU'].mean()
    corrected_signal = signal - baseline_val

    # Calculate noise_std_dev from the corrected pre-elution region
    corrected_pre_signal = corrected_signal[pre_elution_mask]
    noise_std_dev = corrected_pre_signal.std()
    if noise_std_dev == 0:
        noise_std_dev = 1e-6

    # Calculate mean volume difference for width conversion
    vol_diffs = np.diff(volumes)
    mean_vol_diff = np.mean(vol_diffs) if len(vol_diffs) > 0 else 1.0

    # Ensure mean_vol_diff is strictly positive to avoid division by zero
    if mean_vol_diff <= 0:
        mean_vol_diff = 1e-6

    # Step 3: Detect peaks on corrected signal
    height_threshold = 3 * noise_std_dev

```

```

# Ensure find_peaks returns properties with width info
peaks, properties = find_peaks(corrected_signal, height=height_threshold,
                               width=mean_vol_diff)

valid_peaks = []
valid_props = {}

# Guard check: Verify len(peaks) > 0 and properties keys exist
if len(peaks) > 0 and 'left_bases' in properties and 'right_bases' in
    properties:
    peak_heights = corrected_signal[peaks]
    required_width_samples = (2 * noise_std_dev / peak_heights) /
        mean_vol_diff
    actual_width_samples = properties['right_bases'] - properties['left_bases']
        ]

    # Filter peaks based on width condition
    valid_indices = np.where(actual_width_samples > required_width_samples)[0]

    for i in valid_indices:
        peak_idx = peaks[i]
        valid_peaks.append(peak_idx)
        valid_props[peak_idx] = {
            'left': properties['left_bases'][i],
            'right': properties['right_bases'][i],
            'height': peak_heights[i]
        }

valid_peaks = sorted(valid_peaks)

# Step 4 & 5: Integrate and Calculate Purity
run_peaks_data = []

# Pre-compute 260 baseline correction outside the loop
signal_260 = elution_df['UV_2_260_mAU'].to_numpy()
baseline_260 = pre_elution['UV_2_260_mAU'].mean()
corrected_signal_260 = signal_260 - baseline_260

for idx, peak_idx in enumerate(valid_peaks):
    props = valid_props[peak_idx]
    start_idx = props['left']
    end_idx = props['right']

    # Ensure indices are within bounds
    start_idx = max(0, start_idx)
    end_idx = min(len(elution_df) - 1, end_idx)

    # Use array views for integration
    peak_signal_280 = corrected_signal[start_idx:end_idx+1]
    peak_volumes = volumes[start_idx:end_idx+1]

    # Integrate using scipy.integrate.trapezoid
    area_280 = trapezoid(peak_signal_280, peak_volumes)

    # Use pre-computed corrected 260 signal
    peak_signal_260 = corrected_signal_260[start_idx:end_idx+1]
    area_260 = trapezoid(peak_signal_260, peak_volumes)

    purity_ratio = area_280 / area_260 if area_260 != 0 else 0.0

```



```

start_vol = peak_volumes[0]
end_vol = peak_volumes[-1]

# Step 6: Assign to Fraction
# Use the original DataFrame slice for Fraction ID
peak_segment = elution_df.iloc[start_idx:end_idx+1]

if peak_segment['Fraction_unique_ID'].isna().all():
    assigned_frac = "Continuous"
else:
    # Efficient aggregation
    fraction_sums = peak_segment.groupby('Fraction_unique_ID')['UV_1_280_mAU'].sum()

    if fraction_sums.empty:
        assigned_frac = "Unknown"
    else:
        max_sum = fraction_sums.max()
        candidates = fraction_sums[fraction_sums == max_sum].index.tolist()

        if len(candidates) == 1:
            assigned_frac = candidates[0]
        else:
            # Tie-breaker: lower start volume
            candidate_mask = peak_segment['Fraction_unique_ID'].isin(candidates)
            candidate_segment = peak_segment[candidate_mask]
            min_vols = candidate_segment.groupby('Fraction_unique_ID')['volume_ml'].min()
            best_frac = min_vols.idxmin()
            assigned_frac = best_frac

run_peaks_data.append({
    'Peak_ID': f"P{idx+1}",
    'Start_Volume': start_vol,
    'End_Volume': end_vol,
    'Area_280': area_280,
    'Area_260': area_260,
    'Purity_Ratio': purity_ratio,
    'Associated_Fraction_ID': assigned_frac,
    'run_no': run_no
})

# Step 7: Output Handling
if len(run_peaks_data) > 15:
    # Use heapq.nlargest for Top-K selection
    top_15 = heapq.nlargest(15, run_peaks_data, key=lambda x: x['Area_280'])
    omitted_count = len(run_peaks_data) - 15
    md_peaks = top_15
    csv_peaks = run_peaks_data
    note = f"{omitted_count} additional peaks omitted"
else:
    md_peaks = run_peaks_data
    csv_peaks = run_peaks_data
    note = ""

results_list.extend(csv_peaks)

```

```

if md_peaks:
    md_table = "|_Peak_ID_|_Start_Volume_|_End_Volume_|_Area_280_|_Area_260_|_Purity_Ratio_|_Associated_Fraction_ID_|_n"
    md_table += "
    |-----|-----|-----|-----|-----|-----|-----|
    n"

    for p in md_peaks:
        md_table += f"|_{p['Peak_ID']}|_{p['Start_Volume']:.2f}|_{p['End_Volume']:.2f}|_{p['Area_280']:.2f}|_{p['Area_260']:.2f}|_{p['Purity_Ratio']:.2f}|_{p['Associated_Fraction_ID']}|_n"

    header_line = f"###_Run:_{run_no}"
    if note:
        header_line += f"|_{note}"

    md_content.append(header_line + "\n")
    md_content.append(md_table)
    md_content.append("\n")

# Save CSV
if results_list:
    df_results = pd.DataFrame(results_list)
    df_results.to_csv(OUTPUT_CSV, index=False)

# Save MD
if md_content:
    with open(OUTPUT_MD, 'w') as f:
        f.write("#_Peak_Metrics_Summary\n\n")
        f.writelines(md_content)

print("Step_2_completed.")

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os
from scipy.signal import find_peaks

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# 1. Load the raw chromatography data
input_file = '/inputs/chromatography_combined.csv'

# Verify File Path
if not os.path.exists(input_file):
    raise FileNotFoundError(f"Input_file_not_found:_{input_file}. Cannot_proceed.")
)

df_raw = pd.read_csv(input_file)

# Verify required raw columns exist
required_raw_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU']
# Handle potential column name variations (e.g., 'run' vs 'run_no')
if 'run_no' not in df_raw.columns and 'run' in df_raw.columns:

```

```

df_raw.rename(columns={'run': 'run_no'}, inplace=True)

if not all(col in df_raw.columns for col in required_raw_cols):
    raise ValueError(f"Missing required raw columns in {input_file}. Expected: {required_raw_cols}, Found: {df_raw.columns.tolist()}")

# 2. Perform Peak Detection and Integration
processed_rows = []

# Sort by run and volume to ensure correct integration order
df_raw = df_raw.sort_values(by=['run_no', 'volume_ml'])

for run_id, group in df_raw.groupby('run_no'):
    # Extract x (volume) and y (signal)
    x = group['volume_ml'].values
    y = group['UV_1_280_mAU'].values

    # Normalize signal to handle baseline variations if necessary, but find_peaks
    # works on raw
    # Use find_peaks to detect local maxima
    # height parameter can be adjusted based on noise level; using a small
    # fraction of max for robustness
    height_threshold = 0.05 * np.max(y) if np.max(y) > 0 else 0
    peaks, properties = find_peaks(y, height=height_threshold, distance=5)

    if len(peaks) == 0:
        continue

    # Calculate Area Under Curve (AUC) for each peak
    # Simple trapezoidal integration between peak boundaries
    # We define boundaries as the midpoint between peaks or start/end of data
    peak_indices = [0] + list(peaks) + [len(y) - 1]

    for i, peak_idx in enumerate(peaks):
        # Determine start and end of the peak region
        if i == 0:
            start_idx = 0
        else:
            prev_peak = peaks[i-1]
            start_idx = int((prev_peak + peak_idx) / 2)

        if i == len(peaks) - 1:
            end_idx = len(y) - 1
        else:
            next_peak = peaks[i+1]
            end_idx = int((peak_idx + next_peak) / 2)

        # Ensure indices are within bounds
        start_idx = max(0, start_idx)
        end_idx = min(len(y) - 1, end_idx)

        # Calculate Area using trapezoidal rule
        peak_area = np.trapz(y[start_idx:end_idx+1], x[start_idx:end_idx+1])

        # Assign Peak_ID relative to the run (e.g., 1, 2, 3...)
        peak_id = i + 1

        processed_rows.append({
            'run_no': run_id,

```

```

        'Peak_ID': peak_id,
        'Area_280': peak_area
    })

# Create the processed DataFrame
df = pd.DataFrame(processed_rows)

# If no peaks were found at all
if df.empty:
    raise ValueError("No peaks detected in the chromatography data. Check input data quality.")

# Filter for valid peaks (Area > 0)
df_valid = df[df['Area_280'] > 0]

if df_valid.empty:
    raise ValueError("No valid peaks found after filtering.")

# Check for duplicate Peak_IDs within a run (should not happen with our logic, but safety check)
if df_valid.groupby(['run_no', 'Peak_ID']).size().max() > 1:
    raise ValueError('Duplicate Peak_IDs found within a run.')

# 3. Calculate Total_Valid_Area per run
run_totals = df_valid.groupby('run_no')['Area_280'].sum()

# 4. Calculate Relative_Proportion for each peak
df_valid['Total_Valid_Area'] = df_valid.groupby('run_no')['Area_280'].transform('sum')
df_valid['Relative_Proportion'] = df_valid['Area_280'] / df_valid['Total_Valid_Area']

# 5. Generate Comparative Analysis (Stacked Bar Chart)
runs = df_valid['run_no'].unique()
peaks = df_valid['Peak_ID'].unique()
num_peaks = len(peaks)

# Prepare data for plotting
pivot_data = df_valid.pivot_table(index='run_no', columns='Peak_ID', values='Relative_Proportion', aggfunc='sum', fill_value=0)

# Determine if we need to group peaks
if num_peaks > 10:
    conditions = [
        df_valid['Relative_Proportion'] > 0.1,
        (df_valid['Relative_Proportion'] > 0.01) & (df_valid['Relative_Proportion'] <= 0.1)
    ]
    choices = ['Major', 'Minor']
    df_valid['Category'] = np.select(conditions, choices, default='Trace')

    agg_data = df_valid.groupby(['run_no', 'Category'])['Relative_Proportion'].sum()
    agg_data = agg_data.unstack(fill_value=0)
    categories = ['Major', 'Minor', 'Trace']
    agg_data = agg_data.reindex(columns=categories, fill_value=0)

    plot_data = agg_data
    legend_labels = categories
    colors = ['#1f77b4', '#ff7f0e', '#2ca02c']

```

```

else:
    plot_data = pivot_data
    legend_labels = peaks
    colors = None

# Create the plot
plt.figure(figsize=(12, 8))
if num_peaks > 10:
    plot_data.plot(kind='bar', stacked=True, ax=plt.gca(), color=colors)
else:
    plot_data.plot(kind='bar', stacked=True, ax=plt.gca())

plt.title('Relative_Compound_Proportions_per_Run')
plt.xlabel('Run_No')
plt.ylabel('Relative_Proportion')
plt.ylim(0, 1.05)
plt.legend(title='Category' if num_peaks > 10 else 'Peak_ID', labels=legend_labels)
plt.tight_layout()

# Save the plot
plt.savefig('/workspace/run_proportion_comparison.png', dpi=300)
plt.close()

# 6. Identify Outliers and Create Summary
run_summary = df_valid.groupby('run_no').agg(
    Total_Valid_Area=('Area_280', 'sum'),
    Number_of_Peaks=('Peak_ID', 'count'),
    Sum_of_Relative_Proportions=('Relative_Proportion', 'sum')
).reset_index()

threshold = 0.05
run_summary['Status'] = np.where(
    np.abs(run_summary['Sum_of_Relative_Proportions'] - 1.0) > threshold,
    'Outlier',
    'Normal'
)

# Prepare Markdown Table
md_header = "|_run_no_|_Total_Valid_Area_|_Number_of_Peaks_|_Sum_of_Relative_Proportions_|_\\n"
md_separator = "
|-----|-----|-----|-----|\\n
"

run_summary['run_no_str'] = run_summary['run_no'].astype(str)
run_summary['Total_Valid_Area_str'] = run_summary['Total_Valid_Area'].map('{:.2f}'.format)
run_summary['Number_of_Peaks_str'] = run_summary['Number_of_Peaks'].astype(str)
run_summary['Sum_of_Relative_Proportions_str'] = run_summary['Sum_of_Relative_Proportions'].map('{:.4f}'.format)

md_rows = (
    run_summary['run_no_str'] + "_|_" +
    run_summary['Total_Valid_Area_str'] + "_|_" +
    run_summary['Number_of_Peaks_str'] + "_|_" +
    run_summary['Sum_of_Relative_Proportions_str'] + "_|"
).tolist()

```

```

max_total_lines = 20
header_sep_lines = 2
note_line = 1

if len(md_rows) > (max_total_lines - header_sep_lines):
    max_data_rows = max_total_lines - header_sep_lines - note_line
    md_rows = md_rows[:max_data_rows]
    truncation_note = "\n*Note: Table truncated to fit 20-line limit.*"
else:
    truncation_note = ""

md_content = md_header + md_separator + "\n".join(md_rows) + truncation_note

# Append to final_summary.md
summary_file = '/workspace/final_summary.md'

if os.path.exists(summary_file) and os.path.isdir(summary_file):
    raise IsADirectoryError(f"Cannot write to {summary_file} because it is a directory.")

if os.path.exists(summary_file):
    with open(summary_file, 'a') as f:
        f.write("\n### Run-Level Normalization Summary\n")
        f.write(md_content)
else:
    with open(summary_file, 'w') as f:
        f.write("# Final Analysis Summary\n\n")
        f.write(md_content)

print("Analysis complete. Files saved to /workspace/")

```

Listing 3: Code_3